

LLMProvider

LLMProvider

Један интерфејс, 43 провајдера — са уграђеним прекидачима струјног кола, поновним покушајима и мониторингом здравља.

Сажетак

LLMProvider је генерички, виšekратно употребљив модул Go који дефинише уједињени интерфејс LLMProvider, заједно са обрасцима отпорности на грешке — прекидачем струјног кола, мониторингом здравља, поновним покушајима са експоненцијалним одлагањем, лењим учитавањем — и испоручује 43 конкретне имплементације провајдера иза тог једног уговора, уз генерички адаптер компатибилан са OpenAI и поштеним, без унапред кодираних резервних решења за откривање модела.

Кратак опис

Вишепут употребљив модул Go који излаже један интерфејс LLMProvider (Complete, CompleteStream, HealthCheck, GetCapabilities, ValidateConfig) заједно са примитивима за толеранцију грешака — прекидачем струјног кола, монитором здравља, поновним покушајима са експоненцијалним одлагањем и насумичним кашњењем, лењом иницијализацијом — преко 43 адаптера провајдера и генеричког адаптера компатибилног са OpenAI. Безбедан за рад у више нити.

Детаљан опис

LLMProvider је апстракциони слој који свака услуга која користи LLM треба, али га готово нико не гради како треба — неугледна инфраструктура која раздваја демонстрацију од система који преживљава контакт са стварним саобраћајем. Дефинише један интерфејс свестан могућности — Complete,

CompleteStream, HealthCheck, GetCapabilities, ValidateConfig — тако да апликативни код циља тачно један уговор, без обзира који од 43 позадинска система одговара на позив, а затим испоручује оперативно учвршћивање које претвара крхке позиве провајдера у нешто што можете покренути у продукцији без задржавања даха. Тростањни прекидач струјног кола (затворен → отворен → полу-отворен) транспарентно обавија било ког провајдера — укључујући и његов *стриминг канал*, где се празан ток исправно рачуна као грешка — тако да један лоше понашајући позадински систем може да активира прекидач и спречи да обори целу услугу; централни CircuitBreakerManager прати све прекидаче одједном. Конфигурабилни монитор здравља континуирано проверава провајдере кроз стања здраво / деградирано / нездраво / непознато на основу провера са праговима и интервалима, тако да се деградација уочава, а не открива тек када дође до прекида у раду. Логика поновних покушаја комбинује експоненцијално одлагање са насумичним кашњењем, доносећи одлуке на основу статуса — понавља грешке вредне понављања (429, 5xx, пролазне мрежне грешке), никада не троши ресурсе на 4xx или отказани контекст — са ограниченим кашњењима како би се спречила олуја одлагања. Лења иницијализација одлаже конструкцију сваког провајдера до његове прве стварне употребе — намерни дизајнерски избор који чини регистрацију свих 43 провајдера практично бесплатном.

Модул испоручује 43 конкретна пакета провајдера уз генерички адаптер компатибилан са OpenAI, који имплементира цео интерфејс за *било који* ендпоинт /v1/chat/completions — аутентификацију путем Беарер токена, SSE стриминг са исправним руковањем [DONE] — тако да провајдер без наменског пакета и даље постаје првокласни грађанин чим адаптер усмерите на његов URL. Креденцијали се разрешавају на тачно једном месту (apiKey, користећи стриктну конвенцију ApiKey_<Provider>), чиме се на извору затвара цела класа грешака типа „хардкодирани кључ пролази тестове, прави кључ није повезан, производ отказује у продукцији”. Откривање модела је намерно, готово упорно поштено: испитује живе API-је провајдера иза кеша са ограниченим трајањем, а — према правилима управљања — стари слој са хардкодираним резервним решењима је потпуно уклоњен. Када откривање уживо не успе, LLMProvider не враћа *ништа* уместо застарелог каталога, тако да позивалац никада не добије ИД модела који изгледа валидно, а затим се не може позвати. Сваки део овог система направљен је да буде безбедан за рад у више нити.

Зашто смо га изградили

Наивни позиви LLM у продукцији отказују — провајдери ограничавају брзину, деградирају или падају, а један лош бацкенд може повући читаву услугу са собом. Каталози модела се мењају, а хардкодиране листе прослеђују позиваоцима ИД-еве који више не раде. LLMProvider централизује интерфејс, обрасце отпорности и поуздано откривање, тако да сваки потрошач аутоматски наслеђује толеранцију на грешке и истинитост.

Зашто је ово револуционарно

Смањује „интеграцију провајдера LLM” на један једини потез — имплементирајте један интерфејс или једноставно усмерите генерички адаптер на крајњу тачку — а затим тај провајдер аутоматски и транспарентно обавијају прекидач кола, праћење здравља и поновни покушаји са експоненцијалним одлагањем и насумичним кашњењем. Отпорност престаје да буде нешто што сваки тим поново измишља (лоше, под притиском рока, након првог пада) и постаје подразумевано понашање библиотеке за свих 43 бацкенда. Инжењеринг поузданости је написан једном, строго тестиран и бесплатно наслеђен од сваког ко га увози.

Шта је иновативно

- **Један интерфејс свестан могућности** — комплетирање, стримовање, здравље, могућности и валидација конфигурације сведени на један уговор који сваки бацкенд поштује на идентичан начин.
- **Транспарентно обавијање прекидачем кола — укључујући стримове.** Прекидач штити канал `CompleteStream`, а не само захтев/одговор, и третира празан стрим као грешку каква заиста јесте — са обавештењима за слушаоце која су безбедна од мртве петље и изван закључавања.
- **43 пакета за провајдере + генерички адаптер компатибилан са OpenAI** — наменски пакети остају лагани, а сваки ненабројани продавац који подржава `/v1/chat/completions` ради чим усмерите адаптер на њега.
- **Јединствени ауторитет за креденцијале (apikeys)** — тачно једно место чита променљиве окружења `ApiKey_<Provider>`, структурално елиминишући несклад „зелени тестови, покварен производ” уместо да само упозорава на њега.
- **Поштено откривање модела (без хардкодираног фаллбацк-а)** — живи API-ји провајдера иза кеша са

временом трајања; у случају грешке враћа nil, никад застарео или измишљен каталог који прослеђује ИД-еве који се не могу позвати.

- **Лења иницијализација са `sync.Once`** — конструкција се одлаже до прве употребе, тако да регистрација свих 43 провајдера готово ништа не кошта док заправо не позовете неког.
- **Антиблүф, вишерегионални Цхалленге стек** — прави покретач који тестира понашање прекидача кола, здравља и поновних покушаја у пет региона, контролисан упареним мутационим тестирањем (немутирани код мора изаћи са статусом 0; убризгана мутација мора форсирати излаз 99), тако да успешно извршен тестни скуп доказиво значи да понашање ради.

Највећи технички изазови и како смо их решили

- **Каскадне грешке провајдера.** Један нестабилан бацкенд не сме повући читаву услугу са собом. Решено тростањим прекидачем кола (затворен → отворен → полуотворен) који транспарентно обавија било ког провајдера *и његов стрим*, отвара се при трајном отказу, испитује опоравак у полуотвореном стању и централно координира `CircuitBreakerManager`.
- **Пролазне грешке и ограничења брзине.** Решено експоненцијалним одлагањем са насумичним кашњењем које је свесно статуса — $\min(\text{PočetnoOdlaganje} \cdot \text{Množilac}^{(n-1)}, \text{MaksimalnoOdlaganje}) \pm \text{nasumičnoKašnjenje}$ — тако да се поновни покушаји распореде уместо да се синхронизују у „громогласно стадо”. Понавља тачно оно што треба поновити (429, 500, 502, 503, 504 и мрежне грешке) и одбија да троши покушаје на отказани контекст или било који други 4xx.
- **Скалирање на много регистрованих, а неискоришћених провајдера.** Са 43 регистрованих провајдера, од којих је само неколико активно у датој услузи, рана иницијализација била би чиста губитак. Решено лењом иницијализацијом заштићеном са `sync.Once`, тако да само провајдери које заправо позовете плаћају трошкове подешавања.
- **Додељивање неважећих ИД-ева модела.** Решено потпуним уклањањем хардкодираног фаллбацк нивоа за откривање (према ЦОНСТ-036) и враћањем ничега при отказу живог откривања — уз дефанзивно копирање при враћању, тако да позивалац не може да мутира кеш или да се утркује са другим читачем. Истинитост је структурално наметнута, а не конвенцијом.
- **Стримовање + исправност конкурентности.** Суптилни начин отказа је мртва петља између закључавања

прекидача и његових повратних позива за слушаоце. Решено снимком слушалаца и обавештавањем ван закључавања са временским ограничењем од 5 секунди, као и откључавањем пре обавештавања при ресетовању — уз све компоненте изграђене за конкурентну употребу и проверене тестовима за утрке (-race скуп).

Технолошки стек

- **Go (1.25.3)** — изабран због врхунске конкурентности, статичких бинарних фајлова и снажне стандардне библиотеке; садржи модул, интерфејс, све примитиве за отпорност и свих 43 адаптера.
- **net/http (стдlib)** — намерно независни ХТТП: покреће клијенте за појединачне провајдере, генерички адаптер компатибилан са OpenAI и позиве за динамичко откривање, тако да нема потребе за ревизијом или ажурирањем трећепартијског транспорта.
- **логрус** — структурирано, нивоом свести логовање тачно тамо где оператерима треба видљивост: унутар промена стања прекидача кола и путање откривања.
- **testify** — покреће тестни сет и, кључно, фиксирање мутационих грана које дају смисао успешном извршавању.
- **yaml.v3** — парсира и18n пакете и конфигурацију у формату који остаје читљив за људе.
- **digital.vasic.models** — заједнички типови LLMRequest / LLMResponse / ProviderCapabilities, смештени на једном месту како би сви адаптери говорили истим речником (документована рунтима зависност).
- **Интерни пакети** — circuit, health, retry, apikeys, discovery, providers/ (43 провајдера + generic) и i18n: површина за отпорност и интеграцију подељена на мале, независно тестиране јединице уместо једног монолитног система.
- **.env + ~/api_keys.sh (конвенција ApiKey_<Provider>)** — јединствен, недвосмислен извор истине за акредитиве, тако да се кључеви повезују на исти начин и у тестовима и у продукцији.
- **Макефиле раце суите (-race -p 1) + Цхалленге покретач** — стуб отпоран на блефирање: детектор трка доказује исправност конкурентности, а Цхалленге покретач тестира стварно понашање кроз хаос, ддос, скалирање, стрес, динамичко откривање и сценарије без пауза.

Статус и напомене о искрености

- **Статус: бета.** Одвојени, поново употребљиви модул; репозиторијум GitHub је јавно доступан.

- **Лиценца: није дефинитивно одређена.** Недоследна — `doc.go` наводи МИТ, док је присутан фајл ЛИЦЕНСЕ у стилу Апацхе-2.0 — проверити пре објављивања.
- `LLMsVerifier` је узводни једини извор истине за канонички каталог модела. Манифест `helix-deps.yaml` изгледа застарео (наводи `deps: []`, док документација тврди да постоји зависност од `digital.vasic.models`); „Тиер 2 (моделс.дев)” у оквиру откривања је планирана, али још неактивна замена.

Приоритетни ниво: `Helix-primary` (кластер LLM-инфраструктуре — одвојени, поново употребљиви модул). Рангира се иза `HelixTrack`.